

DEV-593: Create Redis backup and restore procedure documentation

Department: DevOps

Prepared by: Thanusha Bai V

Date: 17-sept-2026

Task: DEV-587-Redis Security Hardening

1. Executive Summary

This document defines the backup and restore procedure for Glynac's Redis deployments and validates it with a full disaster-recovery drill on a live Redis 8.10.1 instance. The drill covered the complete lifecycle: prepare data → snapshot → copy backup → simulate total data loss → restore → verify. All 8 test keys were successfully recovered from the backup. Two real-world gotchas were discovered and documented: (1) Redis loads AOF in preference to RDB when appendonly yes – a missing AOF directory causes Redis to boot empty even if dump.rdb exists; and (2) restored files must be owned by the redis user with mode 640/660, or Redis refuses to load them. The procedure below is validated end-to-end and ready for Glynac's operational runbook.

2. Backup Mechanisms Overview

Redis has two persistence formats – both must be backed up together:

File	Purpose	Restore role
dump.rdb	Point-in-time snapshot	Fallback if AOF is missing
appendonlydir/ (3 files)	Incremental write log	Primary if appendonly yes

Critical rule: When appendonly yes, Redis loads **AOF first** and ignores RDB. If AOF is missing or corrupt, Redis starts empty – even if a valid dump.rdb exists. **Backups must include both.**

3. Backup Procedure (Validated)

3.1 Trigger a fresh snapshot

```
redis-cli --user admin --pass '<password>' BGSAVE
sleep 2
redis-cli --user admin --pass '<password>' INFO persistence | grep
rdb_last_bgsave_status
```

Expected: rdb_last_bgsave_status:ok. Do not proceed until this shows ok.

3.2 Create a timestamped backup directory

```
BACKUP_DIR=~/.redis-backup-$(date +%Y%m%d-%H%M%S)
mkdir -p "$BACKUP_DIR"
```

3.3 Copy both RDB and AOF

```
sudo cp /var/lib/redis/dump.rdb "$BACKUP_DIR/dump.rdb"
sudo cp -r /var/lib/redis/appendonlydir "$BACKUP_DIR/"
sudo chown -R $USER:$USER "$BACKUP_DIR/"
```

3.4 Verify backup contents

```
ls -lh "$BACKUP_DIR"
ls -lh "$BACKUP_DIR/appendonlydir"
```

Validated result:

```
dump.rdb                                335 B
appendonlydir/appendonly.aof.2.base.rdb 230 B
appendonlydir/appendonly.aof.2.incr.aof 202 B
appendonlydir/appendonly.aof.manifest   102 B
```

3.5 (Optional) Ship off-host

Copy \$BACKUP_DIR to object storage (S3, GCS, Azure Blob) or a remote server:

```
aws s3 cp "$BACKUP_DIR" s3://glynac-redis-backups/ --recursive
```

Never keep the only backup on the same host as Redis.

4. Restore Procedure (Validated)

4.1 Identify the backup to restore

```
BACKUP_DIR=$(ls -dt ~/redis-backup-* | head -1)
echo "Restoring from: $BACKUP_DIR"
ls -la "$BACKUP_DIR"
ls -la "$BACKUP_DIR/appendonlydir"
```

4.2 Stop Redis

```
sudo systemctl stop redis-server
```

Never restore while Redis is running – it may overwrite your files on shutdown.

4.3 Remove existing data

```
sudo rm -rf /var/lib/redis/appendonlydir
sudo rm -f /var/lib/redis/dump.rdb
```

4.4 Restore RDB

```
sudo cp "$BACKUP_DIR/dump.rdb" /var/lib/redis/dump.rdb
```

4.5 Restore AOF

```
sudo cp -r "$BACKUP_DIR/appendonlydir" /var/lib/redis/
```

4.6 Fix ownership and permissions

```
sudo chown -R redis:redis /var/lib/redis/dump.rdb
/var/lib/redis/appendonlydir
sudo chmod 660 /var/lib/redis/dump.rdb
sudo find /var/lib/redis/appendonlydir -type f -exec chmod 640 {} \;
```

Why this matters: Redis runs as the redis user. If restored files are owned by anyone else, Redis silently refuses to load them and boots empty. This was confirmed during the drill.

4.7 Start Redis

```
sudo systemctl start redis-server  
sleep 3
```

4.8 Verify restore

```
redis-cli --user admin --pass '<password>' DBSIZE  
redis-cli --user admin --pass '<password>' GET backup:key1  
redis-cli --user admin --pass '<password>' LRange backup:list 0 -1  
redis-cli --user admin --pass '<password>' INFO persistence | grep -E  
"loading|rdb_last_load_keys_loaded|aof_enabled"
```

Validated result:

DBSIZE	→ 8
GET backup:key1	→ "important-value-1"
LRange backup:list 0 -1	→ item-a, item-b, item-c
rdb_last_load_keys_loaded	→ 5
aof_enabled	→ 1

All 8 keys restored. Redis loaded 5 from RDB, then AOF replay restored the remaining 3.

5. Critical Gotchas (Discovered Live)

5.1 AOF-first loading – a missing AOF dir means an empty Redis

When appendonly yes, Redis **ignores** `dump.rdb` on startup if AOF is unavailable. Restoring only the RDB file (without the AOF directory) results in a **completely empty Redis**, even though the RDB is present and valid.

Rule: Always restore **both** `dump.rdb` and `appendonlydir/`, or explicitly disable AOF in config before starting if you only have RDB.

5.2 Ownership matters – Redis silently refuses foreign files

In the first drill attempt, restored files were owned by ThanushaBai:ThanushaBai. Redis started, but loaded **0 keys**, with no

error in the logs. After `chown redis:redis` and `chmod 640`, Redis loaded all 8 keys.

Rule: After every restore, always run the `chown/chmod` step. Verify `rdb_last_load_keys_loaded > 0` in INFO persistence.

5.3 Permission denied on ls after chown

After `chown redis:redis`, your normal user can no longer `ls` the files. Use `sudo ls` when inspecting restored files. This is expected and healthy.

6. Backup & Restore Runbook (Summary)

Backup – quick steps

1. `BGSAVE + verify status:ok`
2. `Copy dump.rdb and appendonlydir/ to timestamped dir`
3. `chown -R $USER on backup dir`
4. `Ship to off-host storage`
5. `Record timestamp + key count (DBSIZE)`

Restore – quick steps

1. `systemctl stop redis-server`
 2. `Delete current /var/lib/redis/dump.rdb and appendonlydir/`
 3. `Copy both from backup`
 4. `chown -R redis:redis; chmod 660/640`
 5. `systemctl start redis-server`
 6. `Verify DBSIZE matches backup, spot-check keys`
 7. `Check INFO persistence: rdb_last_load_keys_loaded > 0`
-

7. Verification Checklist

#	Check	Expected
1	rdb_last_bgsave_status after backup	ok
2	Backup dir contains dump.rdb	Present, non-zero size
3	Backup dir contains appendonlydir/ (if AOF on)	3 files present
4	Off-host copy exists	S3/remote present
5	After restore, DBSIZE matches pre-disaster count	Equal
6	After restore, spot- check keys return values	Non-nil
7	After restore, rdb_last_load_keys_loa ded > 0	≥1
8	After restore, aof_last_load_status	ok

8. Retention & Scheduling Recommendations

Item	Recommendation
Backup frequency	Hourly for sessions/queues; daily for cache
Retention	7 daily, 4 weekly, 3 monthly
Storage	Off-host (S3/GCS) with versioning + lifecycle
Encryption	Enable at rest (SSE) and in transit (TLS)
Testing	Full restore drill quarterly
Alerts	Alert if a scheduled backup misses or is <1MB

9. Summary

Area	Result
Backup procedure	✅ Validated – 4 files captured (RDB + 3 AOF files)
Disaster simulation	✅ All data wiped, DBSIZE=0 confirmed
Restore procedure	✅ All 8 keys recovered
Gotcha 1	AOF-first loading – missing AOF = empty Redis
Gotcha 2	Ownership/permissions must be fixed (redis:redis, 640)
Gotcha 3	sudo ls needed after chown
Drill outcome	Procedure is production-ready

10. Recommendations for Glynac

1. **Back up both RDB and AOF** – never RDB alone when AOF is enabled.
 2. **Always chown redis:redis and chmod 640/660** after restore.
 3. **Ship backups off-host** – same-host backups are not backups.
 4. **Run a restore drill quarterly** – untested backups are not backups.
 5. **Alert on backup failure** and on `rdb_last_load_keys_loaded == 0` after any restore.
 6. **Document the exact backup path** in the on-call runbook (feeds into DEV-595).
-

This procedure is validated end-to-end against a live Redis 8.10.1 instance. All test outputs are reproduced from the reference lab.

END OF REPORT